Web and Data Engineering

Lecture 05: Programmierung von Web-Systemen in Java

Prof. Dr. Michael Granitzer

Ziel dieser Vorlesung

Nachdem Vorlesung 04 HTTP und REST als **Protokoll und Architekturstil** eingeführt hat, zeigt diese Vorlesung die **serverseitige Implementierung**: Wie verarbeitet ein Java-System eingehende HTTP-Requests, wie verwaltet es Zustand trotz Statelessness, und wie bleibt der Code wartbar, wenn aus einem Servlet fünfzig werden?

> Im Fokus steht die Server-Seite: Container, Servlets, Architektur, Zustandsverwaltung und die Evolution hin zu Frameworks wie Spring.

Leitfragen

Java Web Stack und Laufzeitmodell

Leitfrage: Welche Infrastruktur steckt hinter einer Java-Web-Anwendung — und warum braucht man mehr als nur Java-Code?

Was sehen Sie hier?

Eine Java-Web-Anwendung empfängt HTTP-Requests und liefert HTML-Seiten zurück. Wer steuert das eigentlich?



Antwort: Der Application Server / Servlet-Container

Er übernimmt:

- TCP-Verbindungen verwalten
- HTTP parsen
- Threads bereitstellen
- Lebenszyklus von Java-Komponenten steuern
- Request auf die richtige Klasse weiterleiten

Frage: Was steht zwischen dem Browser und Ihrem Java-Objekt?

Vorgriff: So sieht der Java-Code aus

Damit das ??? nicht abstrakt bleibt — ein vollständiges, minimal lauffähiges Servlet:

```java
package com.example;

import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;
import java.io.IOException;

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest req,
                         HttpServletResponse res)
            throws IOException {
        String name = req.getParameter("name");
        res.setContentType("text/html;charset=UTF-8");
        res.getWriter().println(
            "<h1>Hallo, " + name + "!</h1>");
    }
}
```

Was passiert hier?

- `@WebServlet("/hello")` registriert die Klasse beim Container für die URL `/hello`
- Bei `GET /hello?name=Alice` instanziiert der Container die Klasse und ruft `doGet(req, res)` auf
- `req` enthält Parameter, Header, Body; `res` ist der noch leere Antwort-Stream
- Wir schreiben HTML in `res.getWriter()` — der Container überträgt das als HTTP-Response zurück

> **Genau das ist das** ??? aus der vorherigen Folie: Der **Servlet-Container** (z. B. Tomcat) macht aus rohem TCP eine Methodenaufruf-Kette in Ihren Java-Objekten.

Speaker notes

Diese Folie soll vorab den „aha"-Moment liefern, bevor die Jakarta-EE-Standards und Container-Theorie kommen. Wichtige Punkte für den Vortrag: (1) `jakarta.servlet.*` ist der moderne Namespace — der alte `javax.servlet.*` aus Java EE ≤ 8 ist seit Jakarta EE 9 obsolet. (2) Der Container instanziiert pro Servlet-Klasse genau **eine** Instanz und ruft `doGet/doPost` aus mehreren Threads parallel auf — Felder müssen daher thread-sicher sein. (3) `getParameter("name")` liefert `null`, falls der Parameter fehlt — keine Exception. (4) Sicherheitsfalle im Beispiel: das direkte Einsetzen von `name` in HTML ist eine **XSS-Lücke**; in einer realen Anwendung würde man `name` HTML-kodieren oder eine Template-Engine wie Thymeleaf nutzen. Dieses Beispiel ist bewusst minimal; alle weiteren Folien in diesem Modul vertiefen einzelne Aspekte (Lifecycle, Request/Response-API, Konfiguration, Fehlerbehandlung).



Jakarta EE — der Enterprise-Java-Standard

> **Jakarta EE** (früher J2EE, dann Java EE) ist ein offener Standard für enterprise-fähige Java-Webanwendungen. Es definiert eine Menge von Spezifikationen — Implementierungen liefern Application Server wie Tomcat oder GlassFish.

Eigenschaften

- **N-Tier-Architektur:** Client / Web / Business / Data
- **Webbasiert:** HTTP als Kommunikationsprotokoll
- **Server-zentrisch:** Logik lebt auf dem Server
- **Komponentenbasiert:** standardisierte Bausteine (Servlets, Beans, EJBs)
- **Offen:** spezifiziert, nicht nur eine Bibliothek

Historische Stationen

Name	Jahr
J2EE 1.2	1999
Java EE 5	2006
Java EE 8	2017
Jakarta EE 8	2019 (Eclipse Foundation)
Jakarta EE 11	2024

Seit dem Wechsel zur Eclipse Foundation heißt das Paket `jakarta.*` statt `javax.*`.



Die zentralen Jakarta-EE-Standards

Vier Bausteine genügen, um eine moderne Web-Anwendung zu verstehen:

Standard	Aufgabe	Wo in dieser Vorlesung?
Java Servlets	HTTP-Request-Verarbeitung in Java-Klassen	Modul 2
JSP (JavaServer Pages)	Template-basierte HTML-Erzeugung	Modul 3
JDBC	Relationale Datenbanken ansprechen	hier nur erwähnt
JPA	ORM — Objekte auf Tabellen mappen	Modul 4 (Spring Data)

Weitere Jakarta-EE-Standards (nicht in dieser Vorlesung): EJB (transaktionsgesteuerte Business-Logik), JMS (Messaging), JNDI (Resource-Lookup), JTA (verteilte Transaktionen), JAX-RS (REST-Endpoints), JavaMail, JCA. Ein modernes Spring-Boot-Projekt nutzt Servlets, JPA und gelegentlich JMS — EJB und JCA sind heute selten und gehören in Vertiefungen zu Enterprise-Integration.

Speaker notes

Die Reduktion auf vier Standards ist bewusst pädagogisch — die ursprüngliche Liste mit zehn Standards überfordert in einer 90-Minuten-Vorlesung. Hintergrund zu den ausgelassenen: **EJB** war in den 2000ern das zentrale Modell für transaktionale Business-Logik, ist aber durch Spring weitgehend abgelöst. **JMS** steht für asynchrone Messaging-Systeme (ActiveMQ, RabbitMQ) und ist relevant, sobald man Microservices entkoppelt — Thema in einer Architektur-Vorlesung. **JNDI** ist der Resource-Lookup-Mechanismus für DataSources, der heute meist von Spring Boot abstrahiert wird. **JAX-RS** wäre der RESTful-Pendant zu Servlets — in Spring übernimmt das `@RestController`. Wer Jakarta EE „pur" lernen will, sollte sich JAX-RS ansehen, weil es das Standard-Pendant zu Spring REST ist. Die Auslassung von EJB ist auch eine Aussage: Modern werden Transaktionen über Spring `@Transactional` verwaltet, das unter der Haube JPA + JTA orchestriert, ohne dass Studis das EJB-Container-Modell verstehen müssen.

Application Server: Tomcat vs. GlassFish

Apache Tomcat

- **Servlet-Container** (kein vollständiger EE-Server)
- Unterstützt: Servlets, JSP, WebSocket
- Leichtgewichtig, weit verbreitet
- Wird von Spring Boot eingebettet

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Eclipse GlassFish

- **Vollständiger Jakarta-EE-Application-Server**
- Unterstützt: Servlets, EJB, JMS, JCA, JNDI, …
- Referenzimplementierung für Jakarta EE
- Geeignet für Enterprise-Deployments mit allen EE-Features

Faustregel: Tomcat = einfach, Spring-freundlich. GlassFish/WildFly = voller EE-Standard, mehr Overhead.

Der Java-Web-Stack: Schichten im Überblick

Warum Java für Web-Systeme?

Stärken

- Starkes Typsystem
- Ausgereifte Toolchain (Maven, Gradle, IDE-Support)
- Riesiges Ökosystem (Spring, Hibernate, Apache Commons)
- Bewährt in großen Enterprise-Systemen
- Sehr gute Performance bei langlaufenden Server-Prozessen
- Jakarta EE: standardisierte APIs für Portierbarkeit

Abwägungen

Aspekt	Java	Node.js / Python
Startup-Zeit	langsam (JVM)	schnell
Throughput	sehr hoch	mittel
Typsicherheit	statisch	dynamisch
Enterprise-Reife	sehr hoch	wächst
Lernkurve	steil	flacher

Java dominiert im Enterprise-Backend. Spring Boot hat den Einstieg erheblich vereinfacht.

Mini-Übung: Schichten zuordnen

Aufgabe: Welcher Schicht im Java-Web-Stack gehört jedes der folgenden Code-Fragmente an?

```
// Fragment A
@GetMapping("/products/{id}")
public String showProduct(@PathVariable Long id, Model model) { ... }

// Fragment B
public class OrderService {
    public Order placeOrder(Cart cart, Customer customer) { ... }
}

// Fragment C
public interface ProductRepository extends JpaRepository<Product, Long> {
    List<Product> findByCategory(String category);
}
```

Fragment	Schicht	Begründung
A	Web-Schicht (Controller)	HTTP-Mapping, Request-Parameter, gibt View zurück
B	Service-Schicht	Fachlogik, orchestriert mehrere Entitäten
C	Datenzugriff (Repository)	Datenbankabfrage, JPA-Interface

Jakarta EE definiert den Standard für Java-Webanwendungen: eine Menge von APIs für HTTP, Persistenz, Messaging und Transaktionen. Application Server wie Tomcat oder GlassFish implementieren diese Standards und übernehmen Infrastrukturaufgaben, die kein Entwickler neu erfinden sollte. Der Java-Web-Stack besteht aus klar getrennten Schichten — Web, Service, Datenzugriff — und der Container verbindet sie zur Laufzeit.

Request-Verarbeitung auf dem Server

Leitfrage: Wie verarbeitet ein Java-Server HTTP-Anfragen — von der TCP-Verbindung bis zur HTML-Antwort?

Was passiert nach dem HTTP-Request?

Ein Browser schickt `GET /products/42 HTTP/1.1` an den Server. Was passiert jetzt?

```
Browser
  | TCP-Verbindung
  ▼
Tomcat (Servlet-Container)
  | HTTP parsen, Thread zuweisen
  | URL-Mapping: /products/42 → ProductServlet
  ▼
ProductServlet.doGet(request, response)
  | Parameter lesen, Service aufrufen
  ▼
Response: HTML schreiben
  |
  ▼
Browser empfängt HTML
```

Frage: Wer macht den Schritt von der URL zur Java-Klasse?

Der **Servlet-Container** (Tomcat) übernimmt das URL-Mapping — konfiguriert über `web.xml` oder die `@WebServlet`-Annotation. Ihr Java-Code startet erst bei `doGet()`.

Das Servlet-API: Klassenhierarchie

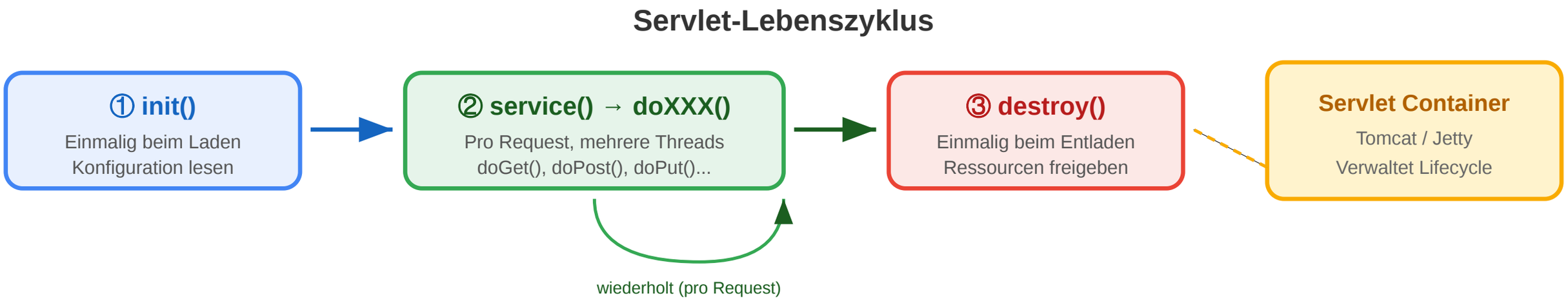
```
jakarta.servlet.Servlet (Interface)
   └──── GenericServlet (abstrakt)
         init(ServletConfig)
         service(ServletRequest, ServletResponse)
         destroy()
          └──── HttpServlet (abstrakt)
                doGet(HttpServletRequest, HttpServletResponse)
                doPost(HttpServletRequest, HttpServletResponse)
                doPut(...)
                doDelete(...)
```

Sie implementieren `HttpServlet` und überschreiben die benötigten `doXXX`-Methoden. `service()` delegiert automatisch an die richtige Methode.

Klasse/Interface	Aufgabe
`Servlet`	Lifecycle-Vertrag
`GenericServlet`	Protokollunabhängig
`HttpServlet`	HTTP-spezifisch
`HttpServletRequest`	Request lesen
`HttpServletResponse`	Response schreiben
`HttpSession`	Session verwalten
`ServletContext`	App-weiter Kontext

> **Multithreading:** Der Container erstellt normalerweise **eine Instanz** pro Servlet und ruft `doGet()`/`doPost()` aus mehreren Threads gleichzeitig auf. Instanzvariablen in Servlets sind daher **nicht thread-sicher**.

Servlet-Lifecycle: init → service → destroy



Thread-Pool im Container:

Thread 1	Thread 2	Thread 3	⚠️ Gleiche Instanz!
GET /products/42	POST /orders	GET /cart	Instanzvariablen geteilt

HttpServletRequest: Eingabedaten lesen

```java
protected void doGet(HttpServletRequest req,
                     HttpServletResponse resp)
        throws ServletException, IOException {

    String query   = req.getParameter("q");
    String page    = req.getParameter("page");
    String[] color = req.getParameterValues("color");

    String agent   = req.getHeader("User-Agent");
    String accept  = req.getHeader("Accept");

    String method  = req.getMethod();
    String uri     = req.getRequestURI();
    String queryStr = req.getQueryString();

    BufferedReader body = req.getReader();
    HttpSession session = req.getSession(false);
}
```

Wichtige Methoden

Methode	Liefert
`getParameter(name)`	Einzelwert als String
`getParameterValues(name)`	Array bei Mehrfachwerten
`getQueryString()`	Roher Query-String
`getHeader(name)`	HTTP-Header-Wert
`getMethod()`	GET, POST, ...
`getRequestURI()`	Pfad ohne Host
`getSession()`	Session (neu anlegen)
`getSession(false)`	Session oder null
`getReader()`	Body als Text
`getInputStream()`	Body als Bytes



HttpServletResponse: Antwort schreiben



```java
protected void doGet(HttpServletRequest req,
                     HttpServletResponse resp)
        throws ServletException, IOException {

    resp.setContentType("text/html; charset=UTF-8");
    resp.setContentLength(1024);
    resp.setStatus(HttpServletResponse.SC_OK);
    resp.setHeader("Cache-Control", "no-cache");

    Cookie c = new Cookie("theme", "dark");
    c.setMaxAge(60 * 60 * 24);
    resp.addCookie(c);

    PrintWriter out = resp.getWriter();
    out.println("<!DOCTYPE html><html><body>");
    out.println("<h1>Produkte</h1>");
    out.println("</body></html>");
}
```

Wichtige Methoden

Methode	Wirkung
`setContentType(type)`	MIME-Type + Charset
`setStatus(code)`	HTTP-Statuscode
`setHeader(k, v)`	Response-Header
`addCookie(c)`	Cookie senden
`getWriter()`	Text-Ausgabe
`getOutputStream()`	Binär-Ausgabe
`sendRedirect(url)`	302-Redirect
`sendError(code, msg)`	Fehler-Response

> `getWriter()` und `getOutputStream()` schließen sich gegenseitig aus. Einmal aufgerufen, ist der andere blockiert.

Konfiguration: web.xml und @WebServlet

Klassisch: web.xml

```xml
<web-app>
  <servlet>
    <servlet-name>ProductServlet</servlet-name>
    <servlet-class>
      com.example.ProductServlet
    </servlet-class>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>ProductServlet</servlet-name>
    <url-pattern>/products/*</url-pattern>
  </servlet-mapping>
</web-app>
```

`<load-on-startup>1</load-on-startup>` erzwingt `init()` beim App-Start statt beim ersten Request.

Modern: `@WebServlet` (Servlets 3.0+)

```java
@WebServlet(
    name = "ProductServlet",
    urlPatterns = {"/products", "/products/*"},
    initParams = {
        @WebInitParam(name = "maxResults", value = "20")
    },
    loadOnStartup = 1
)
public class ProductServlet extends HttpServlet {
    @Override
    public void init(ServletConfig cfg) throws ServletException {
        super.init(cfg);              // wichtig: Eltern-init aufrufen
        String max = cfg.getInitParameter("maxResults");
    }
}
```

`@WebServlet` ersetzt `web.xml` vollständig. Beide Varianten können koexistieren.

Fehlerbehandlung: `sendError` und Logging

```
protected void doGet(HttpServletRequest req,
                     HttpServletResponse resp)
        throws ServletException, IOException {

    String idParam = req.getParameter("id");
    if (idParam == null) {
        resp.sendError(HttpServletResponse.SC_BAD_REQUEST,
                       "Parameter 'id' fehlt");
        return;
    }

    long id;
    try {
        id = Long.parseLong(idParam);
    } catch (NumberFormatException e) {
        logger.error("Ungültige ID: {}", idParam, e);
        resp.sendError(HttpServletResponse.SC_BAD_REQUEST,
                       "ID muss eine Zahl sein");
        return;
    }
```

`sendError()` setzt Statuscode und leitet an die konfigurierte Error-Page weiter. Nach `sendError()` **darf** kein weiterer Output in die Response geschrieben werden.

Inter-Servlet-Kommunikation: RequestDispatcher

```
RequestDispatcher dispatcher =
    req.getRequestDispatcher("/products/detail");
req.setAttribute("product", product);
dispatcher.forward(req, resp);

RequestDispatcher header =
    req.getRequestDispatcher("/common/header");
header.include(req, resp);
```

Methode	Browser-URL	Wann nutzen?
`forward()`	bleibt gleich	MVC: Controller → View
`include()`	bleibt gleich	Kopf/Fuß einfügen
`sendRedirect()`	ändert sich	Nach POST: PRG-Pattern

Attribute-Sichtbarkeit

Scope	Methode
Request	`req.setAttribute("user", user)`
Session	`req.getSession().setAttribute("cart", cart)`
App-weit	`getServletContext().setAttribute("config", cfg)`

Mini-Übung: Welche API-Methode?

Aufgabe: Bestimmen Sie für jede Aufgabe die passende Servlet-API-Methode.

Aufgabe	Methode?
URL-Parameter `?page=3` lesen	?
HTTP-Header `Accept-Language` lesen	?
Content-Type der Antwort auf JSON setzen	?
HTTP-Status 403 mit Fehlermeldung senden	?
Request an `/error/403.jsp` weiterleiten	?
Daten für den nächsten Request im selben Session-Kontext speichern	?
Ein Cookie mit Ablaufzeit 1 Stunde setzen	?

Aufgabe	Methode
URL-Parameter lesen	`req.getParameter("page")`
HTTP-Header lesen	`req.getHeader("Accept-Language")`
Content-Type setzen	`resp.setContentType("application/json")`

Aufgabe	Methode
403 senden	`resp.sendError(SC_FORBIDDEN, "msg")`
Intern weiterleiten	`req.getRequestDispatcher("/error/403.jsp").forward(req, resp)`
Session-Daten speichern	`req.getSession().setAttribute("key", value)`
Cookie setzen	`c.setMaxAge(3600); resp.addCookie(c)`

Servlets sind das Fundament der Java-Web-Programmierung. Der Servlet-Container übernimmt Netzwerk, Threading und Lifecycle — der Entwickler implementiert `doGet()`/`doPost()` und arbeitet mit `HttpServletRequest` und `HttpServletResponse`. Frameworks wie Spring MVC bauen auf exakt diesem Fundament auf und abstrahieren das Boilerplate.

Formulare, Sessions und Zustandsmanagement

Leitfrage: Wie überbrückt man die Zustandslosigkeit von HTTP — und welche Methode ist die richtige für welchen Einsatzfall?

Das Zustandsproblem des Webs

HTTP ist **zustandslos**: jeder Request ist unabhängig. Der Server erinnert sich nicht an vorherige Requests desselben Clients.

```
Request 1:  GET /login   → 200 OK, eingeloggt
Request 2:  GET /profile → ??? Wer bist du?
Request 3:  POST /cart   → ??? Welcher Warenkorb?
```

Problem: Login, Warenkorb, Formulare — all das erfordert, dass der Server den Nutzer über mehrere Requests hinweg erkennt.

Lösung: Zustand muss explizit transportiert oder gespeichert werden. HTTP-Statelessness ist kein Bug — sie ermöglicht Skalierung. Der Zustand ist die Aufgabe der **Anwendung**, nicht des Protokolls.

> **Zustandslosigkeit und Skalierung:** Weil kein Protokoll-Zustand existiert, kann jeder Request von einem anderen Server bearbeitet werden. Load Balancer können Requests beliebig verteilen — solange der Anwendungszustand nicht im Speicher eines einzelnen Servers steckt.

5 Methoden für Session-Tracking

Methode	Funktionsprinzip	Vorteile	Risiken
1. User Authorization	Credentials bei jedem Request mitsenden	Stateless, REST-konform	Credentials im Netz, kein Sitzungskonzept
2. Hidden Form Fields	`<input type="hidden" ...>`	Kein Cookie nötig	Sichtbar im HTML-Quelltext, manipulierbar
3. URL Rewriting	`?jsessionid=XYZ` oder Pfad-Encoding	Funktioniert ohne Cookies	URL wird lang, Session-ID in Logs/History
4. Cookies	Browser speichert Cookie	Automatisch, unsichtbar	Datenschutz, XSS, SameSite
5. Session-Tracking-API	Server-seitiges `HttpSession`-Objekt	Einfach, Java-nativ	Speicherbedarf, Clustering-Komplexität

> **URL-Rewriting** mit `jsessionid` im Query-String ist unsicher: Die Session-ID erscheint in Server-Logs, Browser-History und Referrer-Headers. Moderne Anwendungen vermeiden dies.

> **Stateless-Alternative für moderne SPAs/APIs:** Statt einer Server-Session-ID schicken Clients einen **signierten Token** (typisch **JWT — JSON Web Token**) im `Authorization: Bearer …`-Header mit. Der Server hält *keinen* Zustand — er verifiziert nur die Signatur und liest Identität/Claims

direkt aus dem Token. Vorteil: skalierbar und Microservice-freundlich; Nachteil: Token-Widerruf vor Ablauf ist schwierig. Diese Vorlesung zeigt klassische Server-Sessions; JWT/OAuth 2.0 wird in der Sicherheits-Vertiefung (L07) behandelt.

Cookies: Mechanismus und Code

```java
Cookie themeCookie = new Cookie("theme", "dark");
themeCookie.setMaxAge(60 * 60 * 24 * 30);
themeCookie.setPath("/");
themeCookie.setHttpOnly(true);
themeCookie.setSecure(true);
response.addCookie(themeCookie);

Cookie[] cookies = request.getCookies();
if (cookies != null) {
    for (Cookie c : cookies) {
        if ("theme".equals(c.getName())) {
            String theme = c.getValue();
        }
    }
}
```

Cookie-Attribute

Attribut	Bedeutung
`MaxAge`	Ablaufzeit in Sekunden; -1 = Session-Cookie
`Path`	Cookie nur für diesen URL-Pfad
`HttpOnly`	Kein JavaScript-Zugriff → XSS-Schutz
`Secure`	Nur über HTTPS senden
`SameSite`	CSRF-Schutz

> Ohne `HttpOnly` kann JavaScript den Cookie lesen — ein XSS-Angriff kann damit Session-Tokens stehlen. `HttpOnly` **immer** für Session-Cookies setzen.

Session-Tracking-API: HttpSession

```
protected void doPost(HttpServletRequest req,
                      HttpServletResponse resp)
        throws ServletException, IOException {
    String user = req.getParameter("username");
    String pass = req.getParameter("password");

    if (authService.authenticate(user, pass)) {
        HttpSession session = req.getSession();
        session.setAttribute("user", user);
        session.setAttribute("loginTime", System.currentTimeMillis());
        session.setMaxInactiveInterval(30 * 60);
        resp.sendRedirect("/dashboard");
    } else {
        resp.sendError(HttpServletResponse.SC_UNAUTHORIZED);
    }
}
```

HttpSession-Methoden

Methode	Bedeutung
`getSession()`	Session holen/anlegen
`getSession(false)`	Session holen oder null
`setAttribute(k, v)`	Daten speichern
`getAttribute(k)`	Daten lesen
`removeAttribute(k)`	Daten entfernen
`isNew()`	Neue Session?
`getId()`	Session-ID
`setMaxInactiveInterval(s)`	Timeout in Sekunden
`invalidate()`	Session zerstören

Der Container überträgt die Session-ID automatisch per Cookie (`JSESSIONID`) oder URL-Rewriting, falls Cookies deaktiviert sind.

Variable Visibility: Wo lebt welcher Zustand?

Scope	Klasse	Sichtbar für	Lebt bis
Application	`ServletContext`	Alle Servlets der App	App-Ende / Neustart
Session	`HttpSession`	Alle Servlets im gleichen Session-Kontext	Session-Ablauf / `invalidate()`
Request	`HttpServletRequest`	Alle Servlets, die diesen Request verarbeiten	Request-Ende
Instanz	Servlet-Instanzvariable	Alle Methoden des Servlets	`init()` bis `destroy()`

Instanzvariablen im Servlet sind thread-unsicher. Mehrere Threads rufen `doGet()` gleichzeitig auf derselben Instanz auf. Nur zustandslose Services oder thread-sichere Objekte dürfen als Instanzvariablen gespeichert werden.

JSP: Java-Templates für die View-Schicht

> **JSP = View in MVC.** JavaServer Pages sind Servlet-generierte HTML-Templates. Der Container kompiliert JSP-Dateien automatisch zu Servlets. Geschäftslogik gehört **nicht** in JSP — nur Darstellungslogik.

JSP-Architektur (MVC)

Rolle	Technologie
Model	Java Beans / Entitäten
View	JSP (Template)
Controller	Servlet

Der Controller legt Daten in `request.setAttribute()` ab, leitet per

Java Beans als Model

```
public class ProductBean implements Serializable {
    private String name;
    private double price;
    public ProductBean() {}
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
    public double getPrice() { return price; }
    public void setPrice(double price) { this.price = price; }
}
```

Java Beans: kein No-arg-Konstruktor → JSP kann den Bean nicht instanziieren.

JSP-Syntax: Die drei Scripting-Elemente



Java Beans in JSP: <jsp:useBean>

```
<jsp:useBean id="today" class="java.util.Date" scope="request" />
<p>Datum: <%= today.toString() %></p>

<jsp:useBean id="product" class="com.example.ProductBean" scope="request" />
<p>Produkt: <jsp:getProperty name="product" property="name" /></p>
<p>Preis: <jsp:getProperty name="product" property="price" /></p>
```

Typischer MVC-Ablauf:

```
Product p = productService.findById(42L);
request.setAttribute("product", p);
request.getRequestDispatcher("/WEB-INF/product.jsp")
       .forward(request, response);
```



Mini-Übung: Welche Session-Methode ist am sichersten?

Szenario: Sie implementieren einen Login für einen Online-Banking-Service. Welche Session-Tracking-Methode wählen Sie — und warum?

Methode	Sicher für Banking?	Begründung
User Authorization (Basic)		
Hidden Form Fields		
URL Rewriting (jsessionid)		
Cookies (ohne HttpOnly)		
HttpSession + Secure HttpOnly Cookie		

Methode	Sicher für Banking?	Begründung
User Authorization (Basic)	Nein	Credentials bei jedem Request — Risiko bei MITM
Hidden Form Fields	Nein	Session-ID im HTML-Quelltext sichtbar und manipulierbar
URL Rewriting	Nein	Session-ID in Browser-History, Logs und Referrer-Headers
Cookies (ohne HttpOnly)	Nein	XSS kann Cookie stehlen und Session übernehmen

Methode	Sicher für Banking?	Begründung
HttpSession + Secure HttpOnly Cookie	**Ja**	Cookie mit `HttpOnly`, `Secure`, kurzem Timeout, server-seitige Session

Zusatz: `session.invalidate()` beim Logout ist Pflicht — ein gestohlenes Cookie wird damit sofort wertlos.

> HTTP-Zustandslosigkeit ist kein Fehler — sie ist Voraussetzung für Skalierung. Anwendungen müssen Zustand explizit verwalten: durch Cookies, Server-Sessions oder zustandslose Tokens. Die Java-Session-Tracking-API (`HttpSession`) ist die praktischste Variante für klassische Web-Anwendungen, erfordert aber Sorgfalt bei Thread-Safety, Cookie-Sicherheit und Clustering. JSP ergänzt Servlets um eine View-Schicht — wobei modernes Java-Web-Entwicklung Template-Engines gegenüber eingebettetem Java-Code bevorzugt.

MVC, Templates und Wartbarkeit

Leitfrage: Warum reichen Servlets für große Projekte nicht aus — und welche Architekturmuster lösen die Probleme?

Das Problem mit reinen Servlets

Ein mittelgroßes E-Commerce-Projekt: 50 Servlets, jedes schreibt HTML direkt mit `out.println()`, enthält SQL-Abfragen und Geschäftslogik.

```
protected void doGet(...) throws IOException {
    String sql = "SELECT * FROM products WHERE id=?";
    PreparedStatement ps = conn.prepareStatement(sql);
    ps.setLong(1, Long.parseLong(req.getParameter("id")));
    ResultSet rs = ps.executeQuery();

    double price = rs.getDouble("price");
    if (userIsVip(req)) price *= 0.9;

    out.println("<html><body>");
    out.println("<h1>" + rs.getString("name") + "</h1>");
    out.println("<p>€ " + price + "</p>");
}
```

Was fehlt?

- Kein zentrales Routing
- Keine Trennung von Darstellung und Logik
- SQL in HTTP-Code — nicht testbar
- Copy-Paste statt Wiederverwendung
- Konfiguration verstreut über 50 `web.xml`-Einträge

Lösung: Ein Framework, das Konventionen, Trennung und Infrastruktur liefert — **Spring**.

Spring Framework: Überblick

> **Spring** ist das meistgenutzte Java-Enterprise-Framework. Es verwendet normale Java-Klassen (POJOs) und liefert Infrastruktur durch **Dependency Injection** und **AOP** — ohne Vererbungszwang, modular, testbar.

Spring-Architektur-Module

Gruppe	Module
Core Container	Core, Bean, Context, SpEL
Data Access	JDBC, ORM (JPA), JMS, Transaction
Web	Web-MVC, WebFlux, WebSocket
AOP	Aspect-Oriented Programming

Kernprinzipien

- **Inversion of Control (IoC):** Spring verwaltet Objekte und ihre Abhängigkeiten
- **Dependency Injection:** Abhängigkeiten werden injiziert, nicht selbst erzeugt
- **AOP:** Logging, Security und andere Querschnittsbelange werden getrennt
- **Convention over Configuration:** Spring Boot automatisiert fast alles

Dependency Injection: Das Hollywood-Prinzip

> **IoC / Hollywood-Prinzip:** „Don't call me, I'll call you." Statt dass eine Klasse ihre Abhängigkeiten selbst erzeugt (`new`), injiziert der Spring-Container sie von außen.

Ohne DI (eng gekoppelt)

```
public class OrderService {
    private ProductRepository repo =
        new ProductRepositoryImpl();
    private EmailService email =
        new SmtpEmailService();

    public void placeOrder(Order order) {
        Product p = repo.findById(order.getProductId());
        email.send(order.getCustomerEmail(),
                   "Bestellung erhalten");
    }
}
```

Mit DI (lose gekoppelt)

```
@Service
public class OrderService {
    private final ProductRepository repo;
    private final EmailService email;

    public OrderService(ProductRepository repo,
                        EmailService email) {
        this.repo  = repo;
        this.email = email;
    }

    public void placeOrder(Order order) {
        Product p = repo.findById(order.getProductId());
        email.send(order.getCustomerEmail(),
                   "Bestellung erhalten");
    }
}
```

Spring erzeugt und verwaltet alle `@Service`, `@Repository`, `@Controller`-Objekte automatisch und injiziert Abhängigkeiten beim Start.

AOP: Querschnittsbelange trennen

Problem: Logging, Sicherheitsprüfungen und Performance-Messungen durchziehen alle Methoden — ohne AOP muss jede Methode denselben Boilerplate-Code wiederholen.

Sicherheit mit `@PreAuthorize`

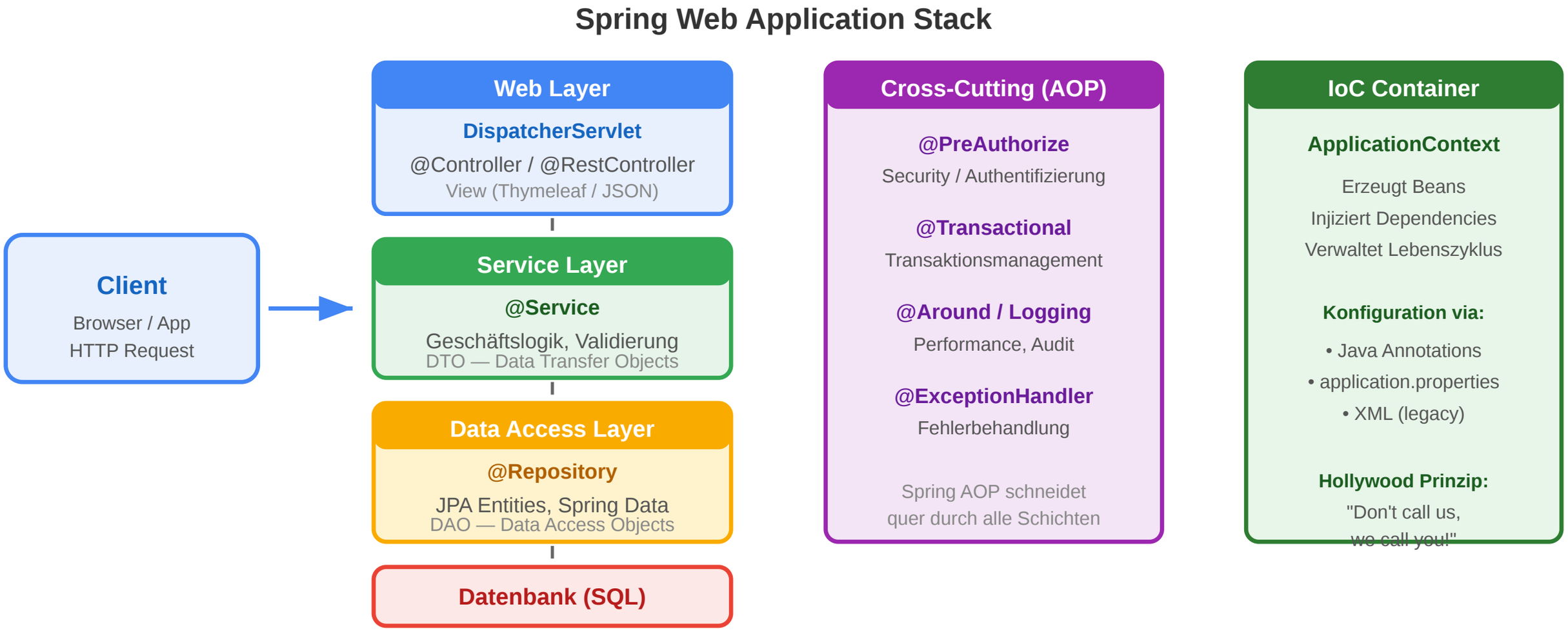
```java
@Service
public class CustomerService {
    @PreAuthorize("#customer.name == authentication.name")
    public Customer updateProfile(
            @P("customer") Customer customer) {
        return repository.save(customer);
    }
}
```

Performance-Logging mit `@Around`

```java
@Aspect
@Component
public class PerformanceAspect {
    @Around("execution(* com.example.service.*.*(..))")
    public Object logTime(ProceedingJoinPoint pjp)
            throws Throwable {
        long start = System.currentTimeMillis();
        try {
            return pjp.proceed();
        } finally {
            long elapsed = System.currentTimeMillis() - start;
            log.info("{} took {}ms", pjp.getSignature(), elapsed);
        }
    }
}
```

> **AOP intercepts method calls.** Spring AOP verwendet Proxies: beim Aufruf einer `@Service`-Methode fängt Spring den Aufruf ab, führt den Aspect aus und ruft dann die eigentliche Methode auf.

Spring Web Stack: Schichten




```java
@Controller
public class ProductController {
    private final ProductService service;
```

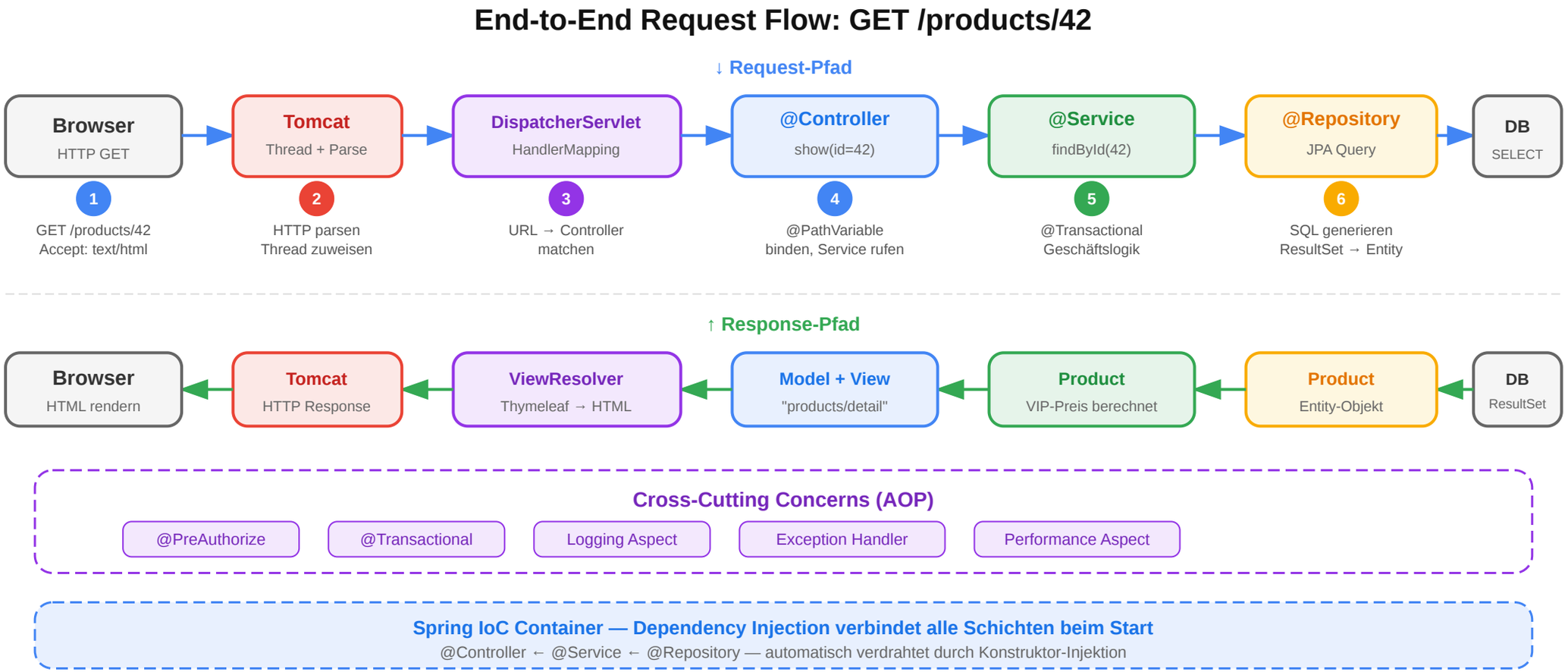
Schicht	Annotation	Aufgabe
Web Layer	`@Controller`	HTTP-Routing, Request-Handling, View-Auswahl
Service Layer	`@Service`	Geschäftslogik, Transaktionen, DTO-Konvertierung
Data Access Layer	`@Repository`	Datenbankzugriff, JPA-Queries
Database	(extern)	Persistenz

```
    @GetMapping("/products/{id}")
    public String show(@PathVariable Long id,
                       Model model) {
        model.addAttribute("product",
            service.findById(id));
        return "products/detail";
    }
}
```

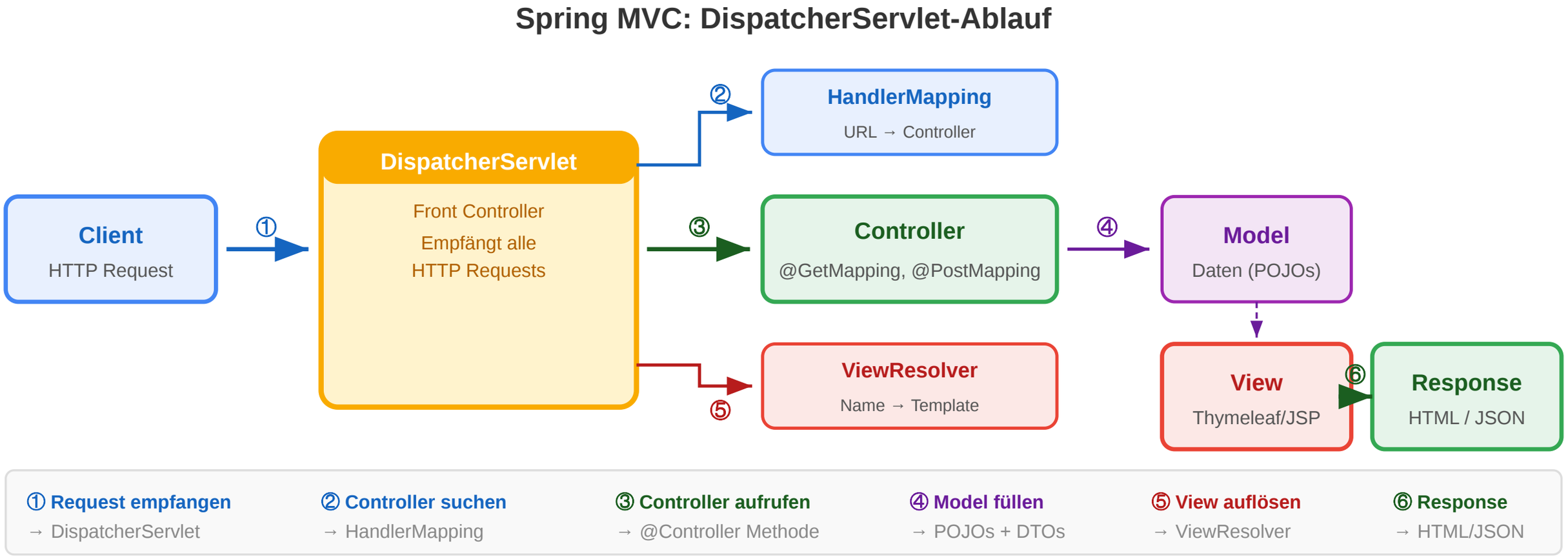
Kommunikationsregeln

- Web Layer → Service Layer
- Service Layer → Repository
- Repository → Datenbank

Spring MVC: End-to-End Request Flow



Spring MVC: DispatcherServlet-Flow



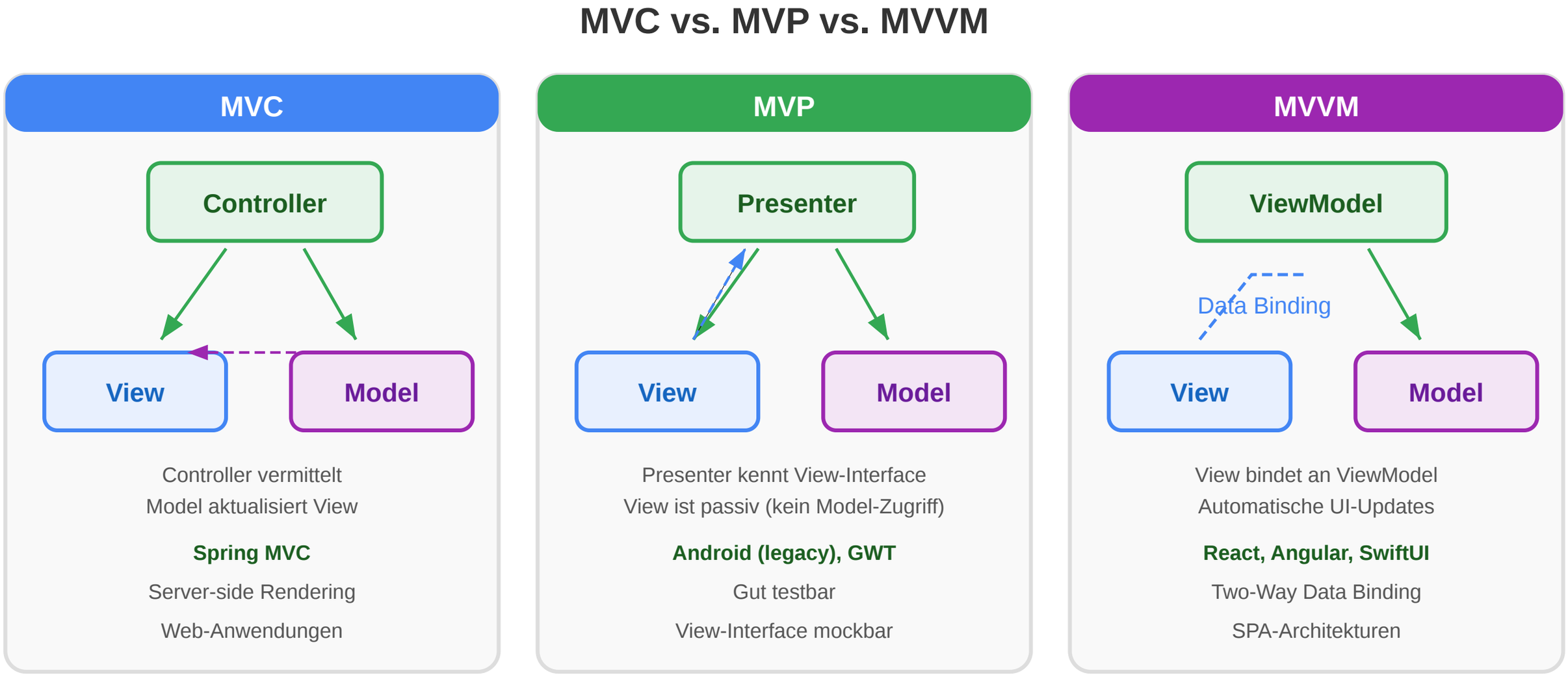

```
Browser
   | HTTP-Request: GET /products/42
   ▼
DispatcherServlet
   | HandlerMapping: welcher Controller?
   ▼
ProductController.show(id=42)
```

DispatcherServlet ist der **Front Controller**: ein einziges Servlet empfängt alle Requests und delegiert an spezialisierte Controller.

```
   | ruft ProductService auf
   | legt Product in Model
   | gibt View-Name zurück
   ▼
ViewResolver
   ▼
Template-Engine rendert HTML
   ▼
HTTP-Response → Browser
```



MVC vs. MVP vs. MVVM



Aspekt	MVC	MVP	MVVM
Wer kommuniziert mit Model?	Controller und View direkt	Nur Presenter	ViewModel (Two-Way-Binding)
View-Logik	View kann Logik enthalten	View ist passiv	View bindet an ViewModel
Testbarkeit	Controller gut testbar	Presenter sehr gut testbar	ViewModel gut testbar
Bindungsrichtung	Unidirektional	Unidirektional	Bidirektional
Typische Verwendung	Spring MVC (Server)	Android Apps	Angular, React, Vue

Wann welches Muster?

Szenario	Empfehlung
Server-seitiges HTML (Spring)	**MVC** mit DispatcherServlet
Rich Client SPA + REST-API	**MVVM** + Spring REST
Android Native	**MVP** oder MVVM

Spring Data JPA: Persistenz ohne Boilerplate

JPA: Object-Relational Mapping

Java-Klasse (@Entity)

```
@Entity
public class Product {

  @Id @GeneratedValue
  private Long id;

  private String name;

  private double price;

  @ManyToOne
  private Category cat;

}
```

JPA / Hibernate

automatisches Mapping →

← Abfragen (JPQL / Criteria)

Datenbank-Tabelle

PRODUCT

ID	BIGINT (PK, AUTO)
NAME	VARCHAR(255)
PRICE	DOUBLE
CAT_ID	BIGINT (FK → CATEGORY)

Vorteile ORM:

- Kein SQL-Code im Java
- Schema aus Klassen generierbar
- Automatisches Caching (L1/L2)

@Entity, @Id, @ManyToOne, @OneToMany, @Column — JPA-Annotationen steuern das Mapping

JPA Entity

Spring Data JPA Repository

```java
@Entity
@Table(name = "products")
public class Product {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "product_name", nullable = false,
            length = 200)
    private String name;

    @Column(nullable = false)
    private double price;
}
```

```java
@Repository
public interface ProductRepository
        extends JpaRepository<Product, Long> {

    List<Product> findByCategory_Name(String catName);
    List<Product> findByPriceLessThan(double maxPrice);

    @Query("SELECT p FROM Product p WHERE p.price < :max")
    List<Product> findCheapInCategory(
            @Param("max") double maxPrice);
}
```



JPA Inheritance Strategies

```java
@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
public abstract class Payment {
    @Id @GeneratedValue Long id;
    double amount;
}

@Entity
@Inheritance(strategy = InheritanceType.JOINED)
public abstract class Animal {
    @Id Long id;
    String name;
}
```

Strategie	Tabellen	Vorteile	Nachteile
SINGLE_TABLE	1	Einfach, schnell	NULL-Spalten
JOINED	1 + N	Normalisiert	JOIN bei jedem Query
TABLE_PER_CLASS	N	Kein JOIN nötig	Polymorphe Queries schwierig

Faustregel: `SINGLE_TABLE` für einfache Hierarchien, `JOINED` für normalisierte Schemas, `TABLE_PER_CLASS` selten.

Mini-Übung: Welche Schicht?

Aufgabe: Ordnen Sie jedes Code-Fragment der richtigen Schicht im Spring-Web-Stack zu (Web, Service, Repository, Entity).

```java
// Fragment 1
@GetMapping("/orders/{id}")
public ResponseEntity<OrderDTO> getOrder(@PathVariable Long id) {
    return ResponseEntity.ok(orderService.findById(id));
}

// Fragment 2
@Transactional
public Order createOrder(Cart cart, Long customerId) {
    Customer customer = customerRepo.findById(customerId).orElseThrow();
    Order order = new Order(customer, cart.getItems());
    return orderRepo.save(order);
}

// Fragment 3
List<Order> findByCustomer_IdAndStatusOrderByCreatedAtDesc(
        Long customerId, OrderStatus status);

// Fragment 4
@Entity @Table(name = "orders")
```

Fragment	Schicht	Hinweis
1	Web Layer	HTTP-Mapping, ResponseEntity
2	Service Layer	`@Transactional`, Geschäftslogik

Fragment	Schicht	Hinweis
3	Data Access Layer	Spring Data Query-Method-Name
4	Entity	JPA-Annotation, Datenbankstruktur

Reine Servlets lösen das HTTP-Problem, skalieren aber schlecht für große Projekte: fehlende Trennung, kein zentrales Routing, viel Boilerplate. Spring löst das durch **Dependency Injection**, **AOP** und den **DispatcherServlet-Front-Controller**. Spring Data JPA eliminiert den Persistenz-Boilerplate durch automatisch generierte Queries. Das Ergebnis: eine Architektur, in der Controller, Service und Repository klar getrennte Verantwortlichkeiten haben.

Wrap-Up

- **Java-Web-Stack:** Container (Tomcat/GlassFish) übernimmt Netzwerk, Threading und Lifecycle; Jakarta EE standardisiert die APIs für HTTP, Persistenz, Messaging und Transaktionen.
- **Servlet-Modell:** `HttpServletRequest`/`HttpServletResponse` sind das Fundament der Java-Web-Programmierung — Spring MVC baut direkt darauf auf.
- **Zustandsmanagement:** HTTP-Statelessness ist kein Bug, sondern Skalierungsvoraussetzung. Cookies, `HttpSession` und Tokens sind die Werkzeuge, um Zustand kontrolliert zu überbrücken — mit Sorgfalt bei Thread-Safety, `HttpOnly`, `Secure` und `SameSite`.
- **Architekturmuster:** Reine Servlets skalieren schlecht für große Projekte. Spring löst das durch Dependency Injection, AOP, den DispatcherServlet-Front-Controller und Spring Data JPA. Das Ergebnis ist eine klare Trennung zwischen Web-, Service- und Repository-Schicht.

Die nächste Vorlesung wendet sich **Data Engineering** zu: Wie skaliert man die Datenhaltung, wenn eine Java-Webanwendung auf Millionen Nutzer und Milliarden Events stößt?

Speaker notes

Im Wrap-Up den Spannungsbogen nochmal aufzeigen: Container → Servlets → Zustand → Framework. Jeder Schritt löst ein konkretes Problem des vorherigen. Servlets lösen das HTTP-Boilerplate, Sessions lösen die Statelessness, Spring löst die fehlende Struktur bei skalierenden Projekten. Dann die Brücke zu L06 bauen: bisher haben wir nur die Anwendungsschicht betrachtet — aber bei großen Systemen wird die Datenhaltung selbst zum Flaschenhals. L06 behandelt Data Engineering: Batch- vs. Stream-Processing, CAP-Theorem, NoSQL, Datenpipelines.

Legacy-Quelle im Kursindex: Kapitel 5: Programmierung von Web-Systemen in Java.